ĐẶC TẢ QUẢN LÝ DỰ ÁN (PROJECT MANAGEMENT SPECIFICATION)

1. Mục đích

Tài liệu này xác định các quy trình, công cụ, vai trò và kế hoạch triển khai trong suốt vòng đời của dự án Thương Mại Điện Tử của nhóm Thủy Lợi N5, nhằm đảm bảo tiến độ, chất lượng sản phẩm và khả năng thích ứng linh hoạt trước những thay đổi trong học phần Công Nghệ Phần Mềm.

2. Mô hình Phát triển Phần mềm: Agile Scrum

Dự án được thực hiện dựa trên mô hình **Agile Scrum**. Đây là khung làm việc linh hoạt giúp phân phối sản phẩm theo từng vòng lặp ngắn, tập trung vào khả năng tương tác, tính thích nghi mạnh mẽ với những yêu cầu thay đổi liên tục và chuyển tải sớm giá trị cốt lõi đến tay người dùng.

2.1 Tại sao chọn Agile Scrum?

- **Khả năng đối phó với sửa đổi yêu cầu liên tục:** Quá trình làm đồ án có thể phát sinh các yêu cầu mới từ giảng viên hướng dẫn hoặc do điều chỉnh thiết kế UI/UX mà không bị hạn chế bởi luồng đi một chiều như mô hình Thác Nước (Waterfall).
- **Deliver early (Mang lại giá trị sớm):** Triển khai Phase 1 với chức năng lõi (Core MVP) xong, ngay lập tức có thể demo đánh giá tính khả thi.
- **Minh bạch tiến độ:** Lịch trình phát triển được thể hiện rõ ràng qua các bảng công việc (Board), các buổi hợp ngắn giúp dễ dàng nhận biết rủi ro sớm.

2.2 Các vai trò trong đội ngũ (Scrum Roles)

Với lợi thế thành viên nhóm, các vai trò được phân bổ như sau:

1. **Product Owner (PO):** Đóng vai trò là đại diện khách hàng/người dùng và định hướng sản phẩm. Xây dựng Product Backlog, làm việc để định nghĩa ra chức năng nào cần được ưu tiên trong Sprint (Giai đoạn này chính là lúc lên list tính năng ở Phase 1, Phase 2).
2. **Scrum Master:** Trưởng nhóm kỹ thuật, đảm bảo toàn đội tuân thủ đúng triết lý Scrum, giúp loại bỏ các "rào cản" (impediments) trong quá trình code (như fix bug cấu hình git, cài cắm env).
3. **Development Team:** Bao gồm các kỹ sư **Frontend (React)** và **Backend (Node.js)**, tự quản lý và quyết định phương pháp hoàn thành Task trong mỗi Sprint.

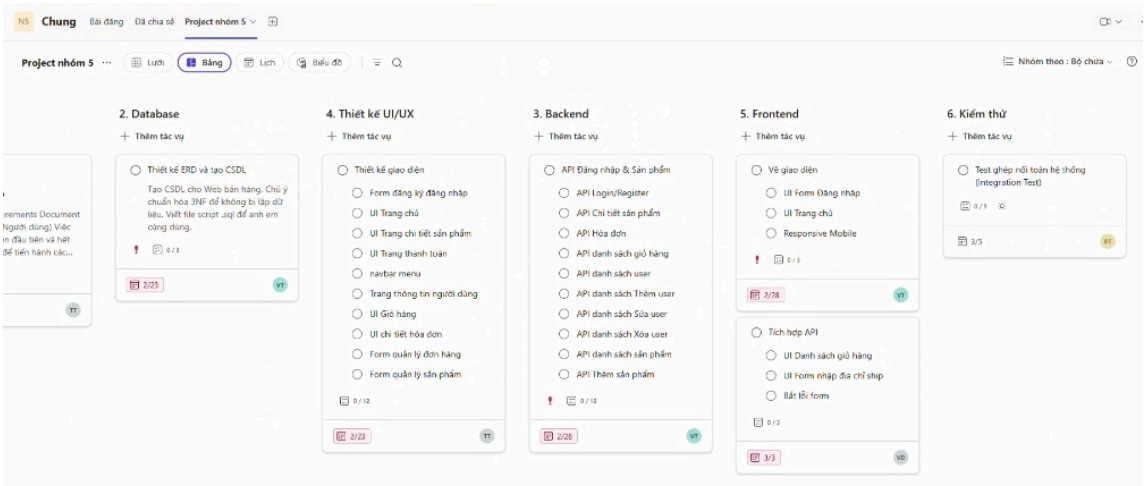
2.3 Các sự kiện Scrum (Scrum Events / Ceremonies)

1. **Sprint Planning (Lập kế hoạch Sprint):** Diễn ra vào đầu mỗi tuần (do tính chất vòng lặp rút gọn của đồ án môn học). Họp thống nhất chọn các Task từ Product Backlog đưa vào Sprint Backlog.
2. **Daily Scrum (Họp nhóm hàng ngày):** Diễn ra ngắn gọn khoảng 15 phút, cập nhật ba trạng thái: Đã làm gì hôm qua, sẽ làm gì hôm nay, và có gặp rào cản lỗi API/bug nào không để Scrum Master hỗ trợ chéo.
3. **Sprint Review (Sơ kết Sprint):** Cuối Sprint, nhóm tích hợp code từ nhánh FE/BE, tiến hành chạy demo sản phẩm thực tế, đánh giá tiến độ so với kế hoạch mảng chức năng.
4. **Sprint Retrospective (Cải tiến Sprint):** Đội ngũ cùng ngồi lại thảo luận để tìm ra những điểm làm tốt và cần cải thiện về cách viết code sạch, cách tổ chức thư mục cho vòng lặp sau.

3. Quy trình Quản lý Công việc & Source Code

3.1 Quản lý Task Board (Microsoft Planner)

- **Công cụ thực tế:** Nhóm sử dụng **Microsoft Planner** làm công cụ quản lý dự án chính. Đây là hệ sinh thái trực quan hỗ trợ phân việc, chia tiến độ trực tiếp, giúp việc phân bổ nhân sự và theo dõi thời hạn các thẻ công việc (Cards) của Sprint trở nên dễ dàng và đồng bộ.
- Công việc được chia nhỏ từ các Use Case thành nhiều Sub-task cụ thể (ví dụ: *Viết API Login*, *Dựng Form UI Component Login*, *Tích hợp API Login vào Redux*).
- Board tiến độ có các cột Bucket chính liên thông quy trình Kanban kéo thả trực quan như: **To Do (Chờ làm)** → **In Progress (Đang code)** → **Done (Đã Review và Test xong)**, thiết lập nhãn (Labels) cho từng loại Bug/Tính năng.



3.1.1 Definition of Done (DoD — Tiêu chí Hoàn thành)

Một Task chỉ được chuyển sang cột **Done** khi đáp ứng đủ tất cả các tiêu chí sau:

#	Tiêu chí	Bắt buộc?
1	Code đã được merge vào nhánh `develop` thành công	✅ Bắt buộc
2	Ít nhất 1 thành viên khác đã review code (Peer Review)	✅ Bắt buộc
3	Không có lỗi nghiêm trọng (crash, 500 error) khi chạy thử	✅ Bắt buộc
4	API endpoint (nếu có) đã được test bằng PowerShell/Postman và trả đúng response	✅ Bắt buộc
5	Tài liệu liên quan (SRS, HLD, Test Cases) đã được cập nhật nếu có thay đổi	🔶 Khuyến nghị
6	Không còn debug `console.log` không cần thiết bị bỏ quên trong code	🔶 Khuyến nghị

3.2 Quy trình Quản lý Mã nguồn (Git / Github)

- Sử dụng mô hình **Git Flow** cơ bản cho quy quy mô nhóm sinh viên:
  - Nhánh `main` : Chứa code đã được Release hoàn chỉnh, sẵn sàng đem đi báo cáo giảng viên.
  - Nhánh `develop` : Nhánh làm việc và hội tụ code cho Sprint hiện tại.
  - Nhánh `feature/...` : Tạo từ develop khi một team member code 1 khối chức năng mới.
- Nguyên tắc: Không push code thẳng lên nhánh `main` . Code xong phải tạo Pull Request (PR) về `develop` để thành viên khác review (tránh đụng chạm conflict file).

3.3 Tiêu chuẩn Viết Code & Đảm bảo Chất lượng (Coding Standards & SQA)

Để đồng bộ nét chữ của tất cả thành viên, dự án áp dụng hệ thống tiêu chuẩn gắt gao nhằm không sinh ra "Rác code":

- **Quy tắc đặt tên (Naming Convention):** Biến (Variables) và Hàm (Functions) dùng kiểu `camelCase` (Ví dụ: `getUserProfile`). Các Tệp màn hình React Component dùng kiểu `PascalCase` (Ví dụ: `ProductCard.js`).
- **Review chéo mã nguồn (Peer Code Review):** Bất cứ tính năng nào code xong cũng cần ít nhất một cộng sự khác đọc lại xem logic có bị trùng số lượng tồn kho hay đặt tên biến tối nghĩa hay không.
- **Tự động bắt lỗi (Linter & Prettier):** Áp dụng bộ gõ tự động uốn nắn cấu trúc (dấu phẩy, dấu cách) để dòng code của 3 kỹ sư 3 máy khác nhau khi gộp lên Github trông như 1 người viết duy nhất.

4. Kế hoạch Triển khai (Roadmap / Sprints Planning)

Dự án được băm nhỏ thành các vòng lặp (Sprints) nối tiếp nhau, mỗi Sprint kéo dài khoảng 1-2 tuần:

4.1 Sprint 1: Thiết lập & Nền tảng Core (1 tuần)

- Hoàn thiện tài liệu URD, SRS, System Design (HLD).
- Setup repository Github, chia nhánh FE (React) và BE (Node.js).
- Cấu hình MongoDB schema cơ sở, thiết lập Redux Store.
- Xác thực đăng nhập Auth JWT rỗng.

4.2 Sprint 2: Hệ sinh thái Sản phẩm (1 tuần)

- Chức năng Admin quản lý CRUD sản phẩm.
- Trang chủ hiển thị danh sách sản phẩm, phân trang, thanh tìm kiếm.
- Click chuyển hướng xem Detail Sản Phẩm.

4.3 Sprint 3: Chu trình Mua sắm (Giỏ hàng & Đơn hàng) (1.5 tuần)

- Xử lý mượt logic thêm, bớt, tăng giảm Item sản phẩm vào Redux Cart.
- Checkout thanh toán.
- Lưu xuống MongoDB Collection `Order` để Admin xem màn hình danh sách đơn.

4.4 Sprint 4: Báo cáo & Đóng gói MVP (Phase 1) (1 tuần)

- Phát triển API Chart thống kê (Tính tổng thu, đếm ngày, tuần, tháng).
- Chức năng Export báo cáo Excel cho Admin.
- Refactor code, Fix bug tồn đọng, UI/UX Polish.
- Build Bundle và đóng băng Source Code cho báo cáo Giai đoạn 1.

4.5 Các Sprint tương lai (Phase 2)

- Focus vào Voucher Engine, Socket Chat, Payment Gateway theo Product Backlog đã đề ra ở tài liệu SRS.

4.6 Bảng Kế hoạch Phát triển & Tiến độ (Development and Milestone Plan)

Dưới đây là bảng chẻ nhỏ lộ trình dài hạn (Long-term Timeline) bắt đầu từ khi nhận đề tài **(21/01/2025)** cho đến khi bảo vệ chung cuộc **(01/03/2026)**.

Thời gian (Timeline)	Giai đoạn (Phase / Sprint)	Các hoạt động chính (Key Activities)	Kết quả / Mốc quan trọng (Milestones)

21/01/2025 - 28/02/2025	Khởi động & Đặc tả Yêu cầu	Thu thập yêu cầu, định nghĩa bài toán. Viết URD, SRS, System Design (HLD). Thiết kế DB (Mongoose).	**Milestone 1:** Hoàn thành toàn bộ Base Documents. Chốt kiến trúc công nghệ (MERN).
01/03/2025 - 30/04/2025	Sprint 1 & 2: Core Platform & Sản phẩm	Khởi tạo Github Base. Code UI/UX khung. Viết API Auth JWT. Chức năng CRUD SP. Đổ Seed Data.	Web chạy thành công luồng đăng nhập, xem danh mục SP. CSDL kết nối thông suốt.
01/05/2025 - 30/06/2025	Sprint 3: Chu trình Mua sắm (Cart & Order)	Build Frontend Giỏ hàng (Redux). Viết logic trừ tồn kho. Kết nối API Đặt hàng (Checkout).	Khách hàng bấm mua thành công, đơn đẩy vào màn hình Quản trị.
01/07/2025 - 31/08/2025	Sprint 4: Báo cáo, Đóng gói MVP (Phase 1)	Vẽ Dashboard KPI Doanh thu (Recharts). Chức năng Export Excel. Tinh chỉnh UI, bắt lỗi Validation.	**Milestone 2 (Release v1.0):** Có sản phẩm demo giữa kỳ với Giảng viên (Minimum Viable Product).
01/09/2025 - 30/10/2025	Đánh giá & Cấu trúc lại (Refactoring)	Thu thập nhận xét từ Giảng viên. Khắc phục Technical Debt. Phân bổ lại cấu trúc Controller/Service.	Mã nguồn sạch sẽ, đạt chuẩn Restful. Tối ưu xong các vòng lặp UI thừa.
01/11/2025 - 31/12/2025	Nâng cấp Phase 2: Thanh toán & Khuyến mãi	Tích hợp cổng thanh toán Sandbox (VNPAY/MoMo). Xây dựng Engine áp mã giảm giá (Vouchers).	Mở rộng hình thức thanh toán ngoài Tiền mặt (COD). Áp tốt được mã giảm giá.
01/01/2026 - 15/02/2026	Nâng cấp Phase 2: System Scale	Cấu hình NodeMailer gửi mail hóa đơn tự động. Bổ sung Ratings & Reviews cho sản phẩm.	**Milestone 3 (Release v2.0):** Một hệ sinh thái E-Commerce hoàn chỉnh. Trải nghiệm người dùng cao.
16/02/2026 - 01/03/2026	Chốt hạ & Báo cáo cuối (Final Defense)	Chạy kiểm thử toàn hệ thống. Họp tổng kết nội bộ phân tích ưu/khuyết. Chuẩn bị Slide PowerPoint, in ấn tài liệu Đặc tả. Diễn tập thuyết trình.	**Milestone 4 (Final):** Tự tin trình bày trước trước Hội đồng đánh giá, bảo vệ mã nguồn.

5. Quản lý Nguồn lực (Resource Management)

Quản lý nguồn lực đóng vai trò thiết yếu để đảm bảo các Sprint diễn ra đúng hạn và các thành viên phát huy tối đa năng lực.

5.1 Nguồn lực con người (Human Resources)

Dự án được phân bổ dựa trên thế mạnh của từng thành viên trong nhóm Thủy Lợi N5.

- **Backend Developer (1-2 người):** Chịu trách nhiệm thiết kế CSDL MongoDB, viết RESTful API bằng Node.js/Express, xử lý logic thanh toán, thống kê và Authentication.
- **Frontend Developer (1-2 người):** Phụ trách cắt HTML/CSS từ UI/UX, tích hợp ReactJS, quản lý global state bằng Redux và ghép nối API từ Backend.

- **Tester / QA (1 người - kiêm nhiệm):** Chịu trách nhiệm viết kịch bản test case, test luồng mua hàng thực tế và báo lỗi lên Trello/Jira để DEV fix bug.
- **Project Manager / Scrum Master:** Theo dõi tiến độ tổng thể, quản lý Source code Git, đánh giá tiến độ sau mỗi Sprint.

5.2 Nguồn lực Thời gian (Time Allocation)

Dự án được giới hạn trong thời gian học kỳ môn Công Nghệ Phần Mềm (ước tính khoảng 4-5 tuần cho Phase 1).

- **Tuần 1:** Tập trung phân tích yêu cầu (URD, SRS), vẽ Use Case, thiết kế Database. Ước lượng: 20 giờ làm việc/tuần.
- **Tuần 2-3:** Giai đoạn Coding cường độ cao (Phát triển API, dựng UI Component). Ước lượng: 30-40 giờ làm việc/tuần cho toàn team.
- **Tuần 4:** Ghép nối API (Integration), kiểm thử luồng Checkout và Fix bug. Ước lượng: 25 giờ làm việc/tuần.
- **Tuần 5:** Hoàn thiện Dashboard, xuất báo cáo, viết tài liệu báo cáo nghiệm thu và chuẩn bị slide thuyết trình. Ước lượng: 15 giờ làm việc/tuần.

5.3 Nguồn lực Phần cứng & Môi trường Thiết bị (Hardware/Equipment)

Do tính chất là phần mềm chạy trên nền tảng Web, yêu cầu về phần cứng không quá khắt khe nhưng cần sự đồng bộ:

- **Máy trạm phát triển (Workstation):**
  - Laptop cá nhân của các thành viên. Khuyến nghị cấu hình tối thiểu: RAM 8GB, CPU Core i5 (hoặc tương đương), Ổ cứng SSD (để cài đặt NodeJS, Docker, MongoDB Compass mượt mà).
  - Hệ điều hành: Windows 10/11, macOS, hoặc Linux (Ubuntu).
- **Môi trường Cloud & Server (Triển khai dự kiến):**
  - **Database Hosting:** Sử dụng **MongoDB Atlas** (gói Free Cluster) để lưu trữ CSDL chung online, giúp mọi thành viên đều gọi được API mà không cần chạy DB Local.
  - **Dịch vụ lưu trữ ảnh:** Cloudinary (dùng để lưu ảnh sản phẩm dưới dạng chuỗi URL thay vì lưu file vật lý nặng nề xuống server).
  - **Môi trường Test:** Các API được test cục bộ bằng **PowerShell (** `Invoke-RestMethod` **)** và **Postman / ThunderClient**, kiểm thử giao diện trên các trình duyệt phổ biến (Google Chrome, Microsoft Edge). Trong giai đoạn SQA nâng cao, nhóm sử dụng PowerShell để gửi HTTP request trực tiếp kiểm tra hành vi API mà không qua frontend.

6. Quản trị Rủi ro (Risk Management Specification)

Trong quá trình phát triển dự án phần mềm theo mô hình Agile, việc nhận diện, đánh giá và xây dựng phương án dự phòng cho các rủi ro là nghiệp vụ cốt lõi để đảm bảo dự án không bị trượt tiến độ (Delay) hay khủng hoảng nguồn lực (Resource Crises).

6.1 Rủi ro về Phạm vi dự án (Scope Creep)

- **Mô tả:** Nhóm dễ rơi vào tình trạng "ảo tưởng sức mạnh" ở giai đoạn đầu, muốn nhồi nhét quá nhiều tính năng phức tạp (Chat realtime, đa cổng thanh toán, AI Suggestion...) trong khi thời lượng môn học rất ngắn. Hậu quả là làm phình to (Over-engineering) kiến trúc hệ thống, code ôm đồm nhưng không chức năng nào chạy hoàn chỉnh.
- **Đánh giá mức độ:** Rất Cao.
- **Chiến lược giảm thiểu:**
  - Thiết lập ranh giới rõ ràng thống nhất với Giảng viên bằng cách chốt **Core MVP (Minimum Viable Product - Sản phẩm khả dụng tối thiểu)** bao gồm: Xem sản phẩm -> Giỏ hàng -> Đặt hàng -> Dashboard Admin.

    - Vận dụng triết lý Backlog trong Scrum: Bất kỳ ý tưởng "hay ho, mỹ miều" nào nảy sinh đều đẩy thẳng xuống nhóm "Phase 2 - Sprint tương lai". Cương quyết từ chối thay đổi phạm vi lõi khi Sprint đang chạy.

6.2 Rủi ro về Quy trình & Phân bổ Thời gian (Timeline Estimation)

- **Mô tả:** Lập kế hoạch thời gian lạc quan tếu (Underestimation). Đồng thời, đồ án thường chạm mặt các kỳ nghỉ lễ kéo dài như Tết Nguyên Đán, 8/3, 30/4 - 1/5. Nhịp điệu của Sprint có thể bị gãy, "nhân sự xõa lễ quên luồng code".
- **Đánh giá mức độ:** Cao.
- **Chiến lược giảm thiểu:**
    - Bơm thêm quỹ thời gian dự phòng **Buffer Time (15-20%)** vào tổng timeline để hấp thụ độ trễ của ngày nghỉ.
    - Sắp xếp các buổi **Release Milestone** ngắt quãng chính xác trước khi kỳ nghỉ bắt đầu (Ví dụ: "Phải đẩy code API giỏ hàng lên nhánh develop trước ngày 28 Tết").
    - Nếu kỳ nghỉ lễ rơi vào giữa một Sprint, Scrum Master chủ động băm nhỏ Sprint đó thành các task cực nhỏ (Micro-tasks) để dứt điểm trong vài ngày, hoặc kéo dài nguyên Sprint đó qua mốc nghỉ.

6.3 Rủi ro về Nhân sự / Đội ngũ (Human Resource Availability)

- **Mô tả:** Trong môi trường đại học, một cá nhân thường gánh nhiều môn học, có lịch thi cuối kỳ chồng chéo, công việc part-time hoặc có việc gia đình. Khi "Nút thắt cổ chai" (Bottleneck) xảy ra – ví dụ Backend Developer bận 1 tuần không viết API ra được khiến Frontend Developer phải ngồi chơi chờ đợi.
- **Đánh giá mức độ:** Cao.
- **Chiến lược giảm thiểu:**
    - **Tách ghép nối (Decoupling):** Khi có giao thức API (Contract), Frontend Team tự động dựng **Mock API** (dùng tệp JSON cứng hoặc thư viện Postman Mock Server/MirageJS) để bôi trơn quy trình render UI mà không cần chờ Backend thật sống dậy.
    - **Phân tản rủi ro tri thức (Cross-functional sharing):** Đảm bảo ít nhất 2 người hiểu được cấu trúc DB để nếu người chính vắng mặt, người phụ vẫn review code và vận hành tiếp được dự án.

6.4 Rủi ro kỹ thuật và Công nghệ (Technical & Architectural Risks)

- **Mô tả:** Lựa chọn bài toán/Stack công nghệ vượt quá năng lực thực tiễn. Ví dụ đội làm Web SPA lại chọn kiến trúc Microservices phức tạp hoặc dùng Database đồ thị mà không thành thạo. Learning Curve (đường cong học tập) quá dốc khiến team mất phân nửa thời gian chỉ để học cú pháp.
- **Đánh giá mức độ:** Trung bình.
- **Chiến lược giảm thiểu:**
    - "Chọn vũ khí quen tay": Tuyệt đối bám sát **MERN Stack** (MongoDB, Express, React, Node) vì cộng đồng đông đảo, dễ dàng tra cứu lỗi trên StackOverflow.
    - Tech Lead/Scrum Master phải dựng một bộ khung dự án (Boilerplate) từ đầu Sprint 1, thiết lập sẵn Redux, React Router, cấu trúc Mongoose Schema để các thành viên khác chỉ việc ghép logic vào form mà không cần xây từ số không.